

Assignment 3

Min Zhu

May 5, 2026

1 Gradient Computation Check

1.1 Correctness of Convolution

In this assignment, a convolution layer is introduced into the model. The convolution operation is implemented using three different approaches: a basic loop-based implementation, a matrix multiplication formulation, and an Einstein summation (einsum) method.

To verify correctness, two levels of consistency checks are performed:

- **External validation:** The output of the loop-based implementation is compared against the provided ground-truth reference in `debug_info.npz`.
- **Internal consistency:** The outputs of the three implementations (loop, matrix multiplication, and einsum) are compared against each other to ensure numerical agreement.

The result of this double check method is shown as below:

```
conv_outputs shape: (8, 8, 2, 5)
conv_gt shape: (8, 8, 2, 5)
Max absolute error: 0.0
```

(a) Loop V.S. torch

```
err(mat vs slow): 1.7763568394002505e-15
err(ein vs mat): 8.881784197001252e-16
err(ein vs slow): 1.7763568394002505e-15
```

(b) Internal Check

Figure 1: Convolution Correctness Check

The consistency across all implementations confirms the correctness of the convolution layer. And a correctly implemented convolution operation is a fundamental prerequisite for obtaining valid gradients.

1.2 Gradient Computation Check

Parameters computed during the forward and backward propagation process are also verified through two methods:

- **Compare with load_data:** The parameters are recorded and compared with the corresponding values in `debug_info.npz` to check whether the implementation matches the expected reference results.
- **Compare with PyTorch result:** The results are further compared with those obtained from a PyTorch-based implementation using for-loops.

```
conv_flat      shape=(128, 5), err=0.000e+00
X1             shape=(10, 5), err=0.000e+00
p             shape=(10, 5), err=2.776e-17
grad_Fs_flat   shape=(48, 2), err=1.110e-16
```

(a) Compare with Load data

```
[Torch gradient check]
w2      shape=(10, 10), abs_err=1.382e-11, rel_err=1.223e-11
b2      shape=(10, 1), abs_err=8.807e-12, rel_err=1.184e-11
w1      shape=(10, 128), abs_err=1.148e-11, rel_err=8.957e-12
b1      shape=(10, 1), abs_err=3.850e-12, rel_err=1.518e-11
Fs_flat shape=(48, 2), abs_err=9.846e-12, rel_err=1.107e-11
b_conv  shape=(2, 1), abs_err=5.496e-12, rel_err=1.247e-11
```

(b) Compare with Torch

Figure 2: Gradient Check

The small error value in Fig.2 shows that the gradient is computed correctly and hereby confirms the code is bug free.

1.3 Run Time

For the initial three layer ConvNet with parameters in architecture 2, the training time is 26.7s with the test accuracy being 55.95%.

2 Bar charts for 4 network architecture

Under the 4 network architecture, the final test accuracy and training times with short runs are shown in Table 1

Table 1: Accuracy and Time

Arch Number	Arch Parameters	Final Test Acc	Running Time(s)
1	f=2, nf=3, nh=50	51.03%	25.5
2	f=4, nf=10, nh=50	55.95%	26.7
3	f=8, nf=40, nh=50	58.70%	26.5
4	f=16, nf=160, nh=50	56.69%	33.2

The corresponding bar charts are demonstrated in Fig.3. It's obvious that Arch1 has the lowest test accuracy and Arch4 takes the longest time to finish the training process. Arch2 and Arch3 achieve a relatively better performance in both accuracy and run time, thus are selected to do further experiments.

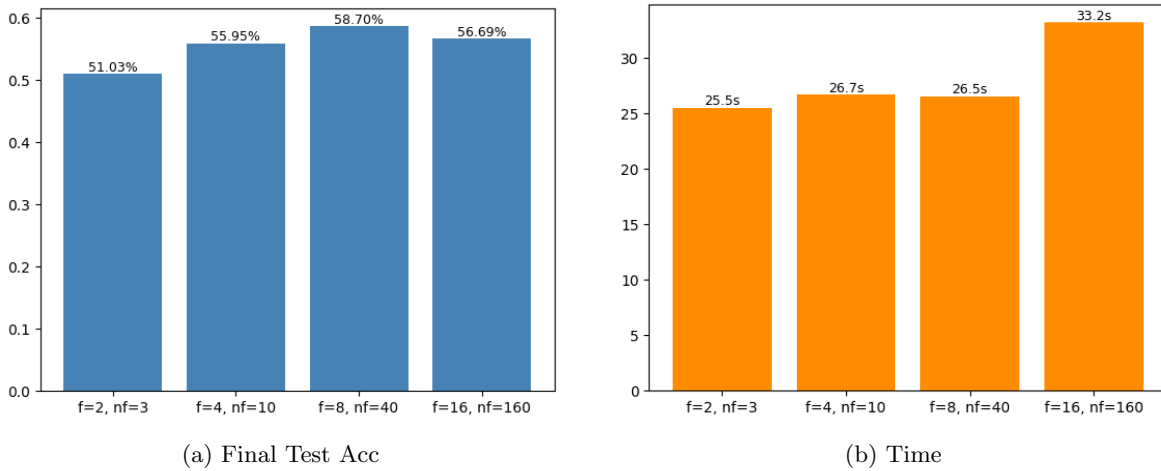


Figure 3: Bar charts under 4 architecture

3 Train for longer

In this section, cyclical learning rates with increasing step sizes are adopted for extended training. With `n_cycle=3` and `step_1=800`, the resulting step sizes are [800, 1600, 3200]. The overall results are in Table 2 and plots of the training and test loss are in Fig.4.

The result indicates that an extension of training updates improves the model performance and arch3 reaches a test accuracy over 60%(up to 61.91%).

To investigate whether layer width significantly affects model performance, a controlled experiment is conducted based on arch2 by modifying the width, resulting in arch5 (f=4, nf=40).

The great improvement(6.22%) from arch2 to arch5 shows that increasing the width of a network enhances model performance.

Table 2: Long Train Result

Arch Number	Arch Parameters	Final Test Acc	Improvement than Short
2	f=4, nf=10, nh=50	58.47%	2.52%
3	f=8, nf=40, nh=50	61.91%	3.21%
5	f=4, nf=40, nh=50	64.69%	-

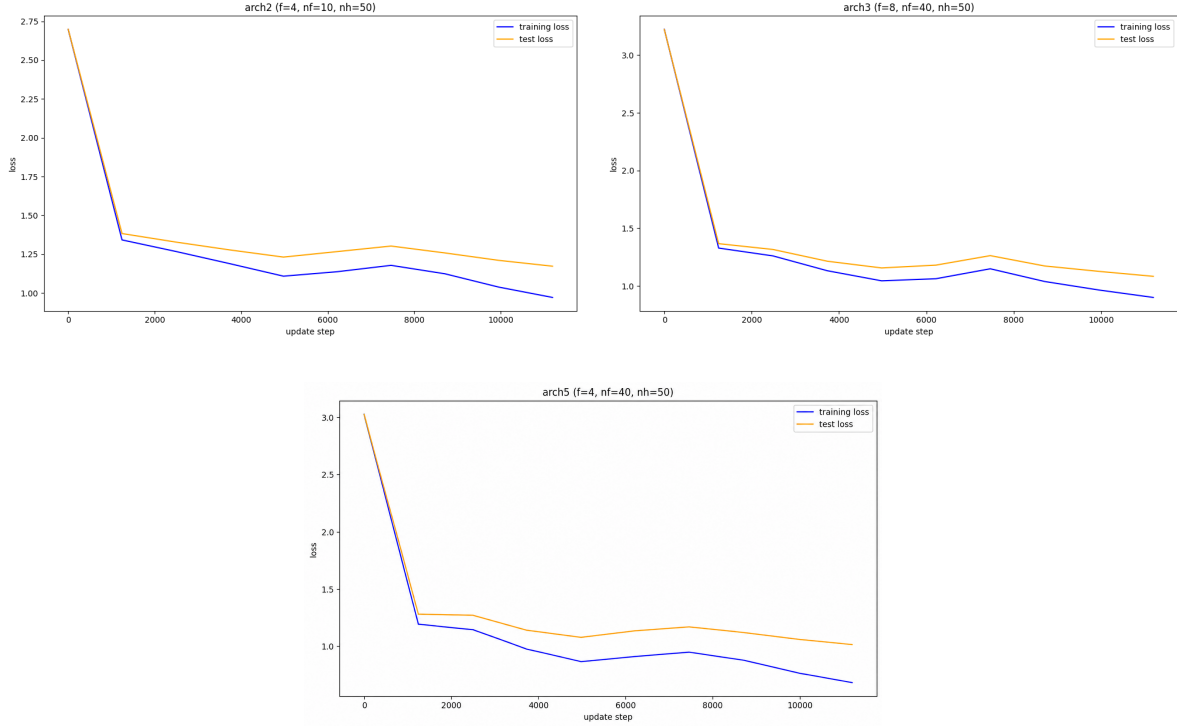


Figure 4: Plots of Training and Test Loss

4 With and without label smoothing

Another way to optimize the network is to increase the width of each layer, which in practice corresponds to increasing the value of `nh`. Under architecture 5: `f=4`, `nf=40`, `nh=300`, the test accuracy grows to **65.70%**.

After adding label smoothing, the test accuracy has a slight rise: from 65.70% to 65.73%. This limited improvement suggests that hyperparameters chosen here are suboptimal and further tuning can be explored to increase the model performance. The training and test loss curves are as below:

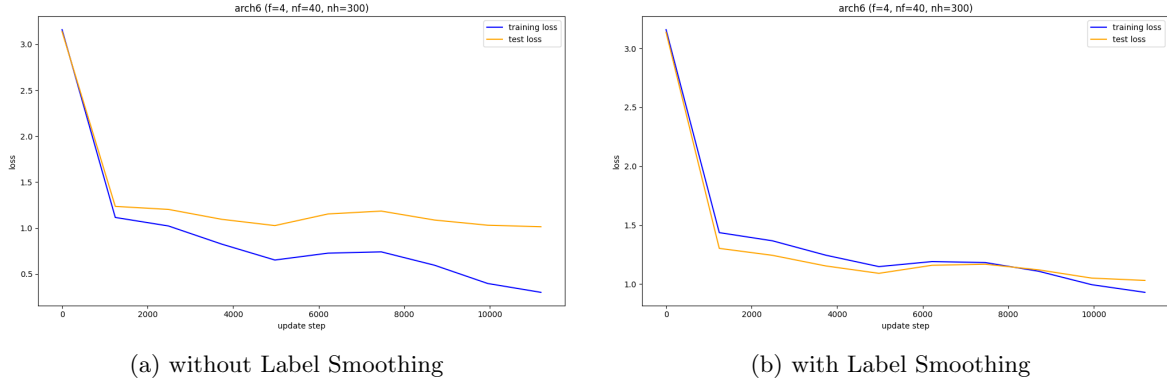


Figure 5: Arch6 - increased width of each layer

Fig. 3(a) shows clear signs of severe overfitting in the later stages of training, as the gap between the training and test loss progressively widens.

After applying label smoothing, the test loss even becomes lower than the training loss. This phenomenon arises because label smoothing modifies the loss definition: during training, the targets are no longer one-hot but softened distributions, which introduce additional uncertainty and effectively increase the training loss. In contrast, the test loss is still computed against hard labels, making it not directly comparable to the training loss and potentially lower.

5 Further steps

5.1 Fine Tuning

Under Architecture 5, the model achieves the highest test accuracy; however, it also exhibits overfitting. The current label smoothing strategy provides only marginal improvement. As a next step, a random search over key hyperparameters, such as the smoothing coefficient λ and the learning rate η can be conducted. With better-tuned hyperparameters, the model performance is expected to improve further.

5.2 Optimizer Switch

The current model is trained using stochastic gradient descent (SGD). This can be further improved by adopting more advanced optimization algorithms, such as SGD with momentum or adaptive methods like Adam. By switching to these optimizers and properly tuning their hyperparameters, the model is expected to achieve faster convergence and improved performance.

5.3 Data Augmentation

Another effective approach to addressing overfitting is to increase the diversity of the training data through data augmentation. After introducing data augmentation such as random cropping, flipping, or translation, the model is exposed to a broader range of input variations, which improves its ability to generalize to unseen data. As a result, the gap between training and validation performance can be reduced, leading to a more robust model.